

Activity 03: Implementing Threads

Sharie Rhea

Southern New Hampshire University

CS 510: Operating Systems Principles and Applications

Brian Enochson

February 22, 2026

The provided Python file, `multithread.py`, creates three threads to perform tasks. The first two threads sum an array of integers by dividing the work in half. Each thread sums one half, then the main thread sums the two results to achieve the final answer. The third thread simply displays basic threading information, such as the thread ID, native ID, and the number of currently running threads. This script utilizes the Python `threading` module, which has support for many platforms and is included by default since version 3.7. Threads are managed by placing each thread into a list. The main function then iterates over this list to first start execution of and then wait for each thread. The `sum_array` function also utilizes a `Lock` object. The lock ensures that only one thread may execute the critical section at a time. The `with` block is syntactical shorthand for calling the `acquire` and `release` methods.

To run the program on your own device, please follow these steps. The `threading` module is supported on Linux, macOS, and Windows. This script has been tested on Linux.

1. Install required dependencies.
 1. Install Python v3.7+.
2. Run the script in your preferred environment.
 1. Integrated development environment (IDE): use the “run button” or conventional way to run a file in your preferred IDE.
 2. Terminal: `python3 multithread.py`
3. Observe the formatted output in the terminal or your IDE’s terminal pane. Run multiple times to see the execution order change due to threading.

Underneath the `threading` module, the operating system will use its own implementation for managing threads. On Linux, this will be through POSIX Pthreads (Silberschatz et al., 2023).

When a thread is created, the operating system creates a new thread control block (TCB) for the thread and assigns it an ID, program counter (PC), and stack pointer (SP) (Silberschatz et al., 2023). This kernel thread may be responsible for one or more user-level threads. However, on Linux this mapping is likely to be one-to-one (Silberschatz et al., 2023).

Developing this script did not pose a significant challenge. To begin, I reviewed the threading module documentation and noted which functions and objects I would likely need, such as `Thread`, `Lock`, `get_ident()`, `start()`, and `join()`. Next, I created the methods for each thread and included simple print statements to check execution. I quickly discovered that the `threading` module does not provide a way for threads to return values. A quick search led me to a workaround: pass a mutable object, like a `list` or `dictionary`, as an argument, mutate it within the thread, and then access it after thread termination. I implemented this behavior and made sure to use a `Lock` object to protect the list during mutation. Finally, I added a sanity check for the `sum_array` result and found that I had an off-by-one error in my array slice indexing. After this fix, I performed one final pass to improve the organization and documentation of the code.

References

Silberschatz, A., Galvin, P. B., & Gagne, G. (2023). *Operating System Concepts* (10th ed.).

Wiley. <https://learn.zybooks.com/zybook/CS-510-10435.202617-1>